MSc Data Science: Coventry University, UK

Artificial Neural Networks

2025.1 BATCH

Name – Perera. P.P.G.S

Username - COMScDS251P-007

31-08-2026

OneDrive Link: COMScDS251P-007 - PPGSPerera (ANN Assignment)

GitHub Repository Link: https://github.com/goyumsamuditha/datasec-uncertainty-aware-sound-classification.git

Table of Contents

Introduction and problem definition 4
Project objectives 4
Question 1: Dataset Overview and Exploratory Data Analysis 5
Dataset source and suitability 5
Dataset structure and target labels 5
Data quality and exploratory analysis 5
Data preparation 7
Error costs and choice of evaluation metric 9
Main findings from EDA and their effect on modelling 9
Question 2: ANN solution design and justification 9
Model choice 9
Input representation and network structure 10
Transfer-learning strategy and parameter count 11
Loss function and optimization 11
Design assumptions and alternatives 11
Why an ANN is appropriate 12
Question 3: Model development, evaluation and optimization 13
Experimental design 13
Majority-class benchmark 13
Classical machine-learning baseline 13
Initial and final ANN 13
Held-out test results 14
Confidence intervals and paired comparisons 15
Hyperparameter optimization and optimizer comparison 16
Training time and inference latency 17
Error analysis 17
Limitations of the evaluation 18
Question 4: Application development and deployment 18
Application overview 18
Prediction and uncertainty display 18
Input validation and practical tests 19
Deployment suitability and limitations 19
Question 5: Explainable AI and model interpretation 20
Explanation methods considered 20
Grad-CAM error examples 21
Sanity and perturbation checks 22
Overall assessment of explainability 23
Question 6: Generative AI/foundation model evaluation 23

Foundation model selected 23
Model configuration and evaluation 24
Performance comparison 24
Practical comparison: performance, latency, cost, and privacy 24
Overall discussion 25
Limitations and future innovation 26
Conclusion 26
References 27

Introduction and problem definition

This project develops a neural network based system for classifying environmental sound recordings. This system is designed as a decision support tool. It identifies likely sound classes, provides explanations and indicates when a prediction should be referred for manual review.

Environmental noise assessment often involves listening to recordings and deciding which sounds are relevant to a particular site or investigation. A sound classification model may help organize this work by suggesting a likely category for each clip. But incorrect prediction can lead to irrelevant clips being retained or an important sound being overlooked. For that reason, this project considers both classification performance and the need to refer uncertain or unfamiliar examples to a person.

The user of this system is an environmental noise assessment officer or acoustic monitoring specialist. For each sound event the officer has to decide whether to exclude it as not belonging to the monitored source or keep it as part of the source. The neural network gives a class, a measure of how sure it is, and an explanation, so the officer can make that decision faster

The primary task predicts 22 main environmental sound classes. DataSEC also contains 287 subclasses, supporting more detailed classification where subclass labels are available.

This report follows the order of the six assessed questions. Question 1 describes the data. Question 2 explains the design of the network and the alternatives I considered. Question 3 covers training, comparison, and tuning. Question 4 describes the web application and how it works. Question 5 will look at explanations. Question 6 compares my network with a large pretrained audio text model.

Project objectives

- Prepare and inspect the DataSEC audio dataset, including class balance, clip duration and data integrity.
- Develop a transfer-learning ANN for 22-class environmental sound classification, using the subclass labels as an auxiliary training signal.
- Compare the ANN with a simple benchmark, a classical machine-learning model and a pretrained audio-text foundation model.
- Assess model performance using macro-F1 as the main metric, supported by class-level results and uncertainty measures.
- Provide a simple application that displays predictions, uncertainty and an explanation.
- Examine model behavior on incorrect and unfamiliar audio examples and discuss the limits of the system.

Question 1: Dataset Overview and Exploratory Data Analysis

Dataset source and suitability

This project uses DataSEC, a dataset for sound event classification of environmental noises. , introduced by Fredianelli and colleagues in Scientific Data (2025). This dataset contains 4292 audio samples with a total duration of approximately 187 hours and 26 minutes. The recordings are mono channel WAV files sampled at 44.1 kHz. Each clip is assigned to one of 22 main sound classes and one of 28 more detailed subclasses, where applicable (Fredianelli et al., 2025).

I selected this dataset because it was focuses on environmental sounds such as transport, animals' human activities, machinery, and other sounds. Its hierarchical labels also made it possible to test whether subclass information could help the main classification task. The dataset is relatively small compared with large general audio collections, which makes transfer learning a practical choice.

The dataset was downloaded from the dataSEC release associated with Zenodo record 23097290.

Dataset structure and target labels

The main folder is organised by the 22 main sound classes, with subclass folders used to represent more specific sound types. The primary prediction target is the 22-class label. The 28-subclass label is used as an auxiliary target during training. This arrangement allows the model to learn more detailed distinctions without changing the main user-facing task.

Dataset property	**Observed value**
Audio files	4,292 WAV recordings
Total duration	Approximately 18 hours 26 minutes
Audio format	Mono-channel WAV
Original sample rate	44.1 kHz
Main classes	22
Subclasses	28
Minimum class size	50 clips
Largest class	Voices: 1,900 clips (approximately 44%)
Duration range	Approximately 0.2 to 481.5 seconds

Data quality and exploratory analysis

An inventory and audio integrity check was completed for all recording files. The integrity results recorded all files as readable. Duration analysis showed a wide range, from very short clips to

recordings lasting several minutes. Music clips were all 30 seconds in the inspected dataset. The difference in duration matters because the model uses a fixed-length input, so some information from longer recordings is not presented to the classifier.

The class distribution is strongly imbalanced. Voices accounts for about 44% of all recordings, while the smallest classes contain 50 examples. The largest class is therefore around 38 times the size of the smallest class. A model trained without addressing this imbalance could achieve a reasonable overall accuracy by favouring common classes while performing poorly on less frequent sounds.

The audio EDA also examined duration, root-mean-square amplitude and low-amplitude recordings. These checks were useful for identifying clips that may be unusually short, quiet or difficult to classify. The dataset authors report that the clips were reviewed and processed to focus on individual sound events. However, the original source identity is not consistently preserved in the local files, because source recordings may have been cut, mixed, edited or renamed during dataset preparation.ds.

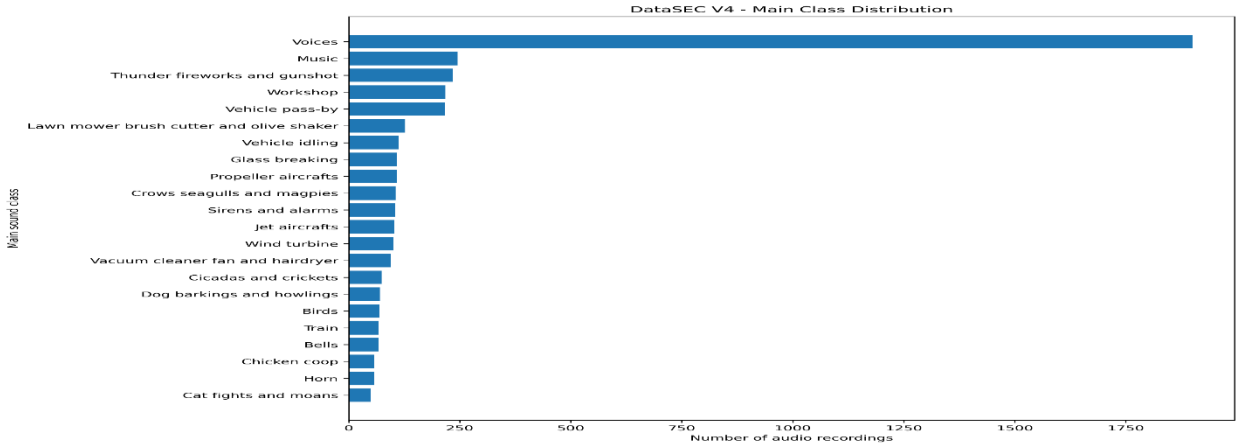
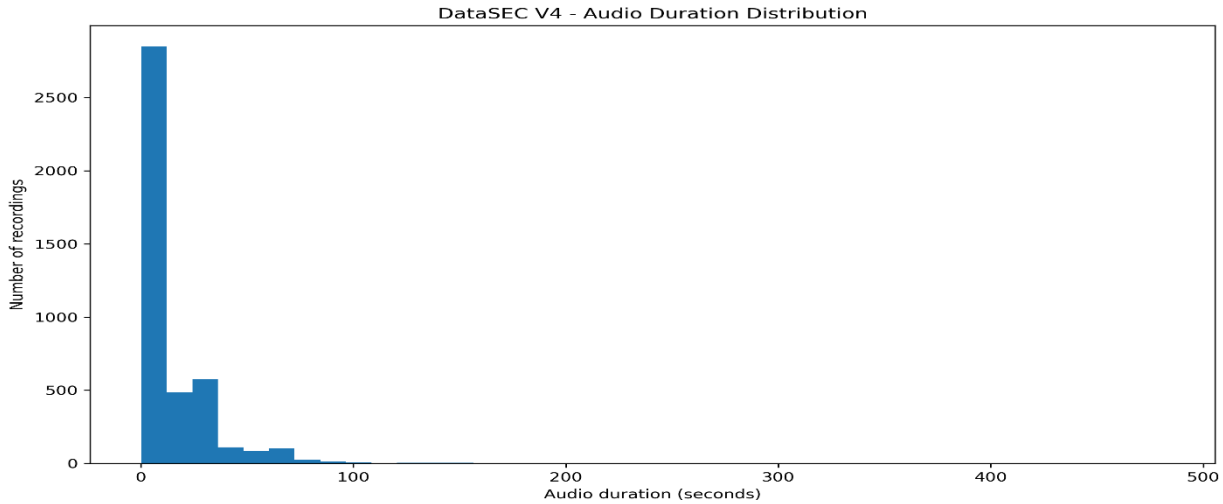


Figure 1. Number of recordings in each of the 22 main DataSEC classes.



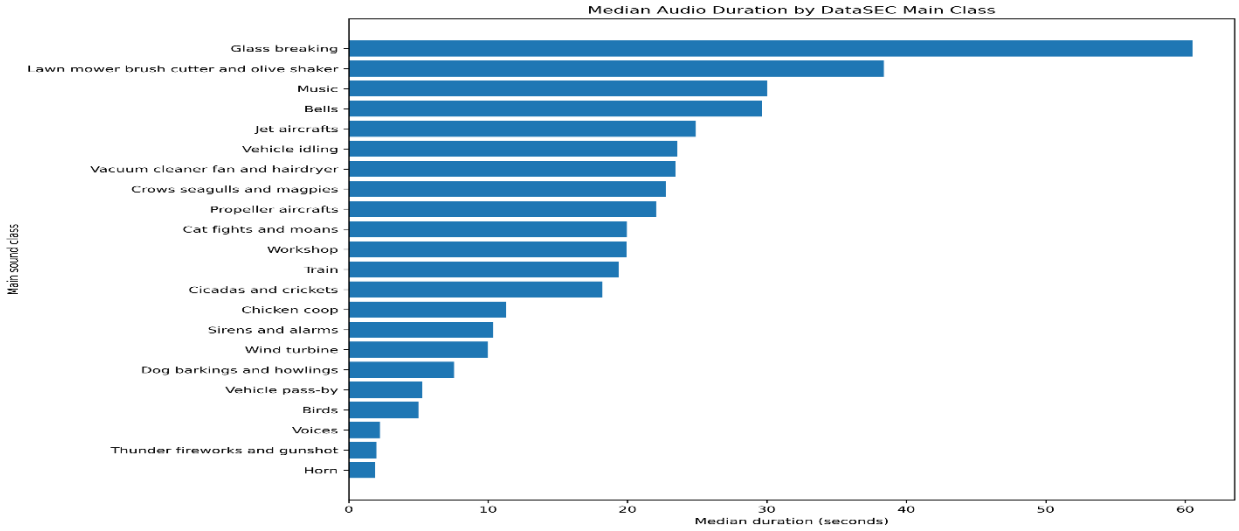


Figure 2. Overall clip-duration distribution and median duration by class.

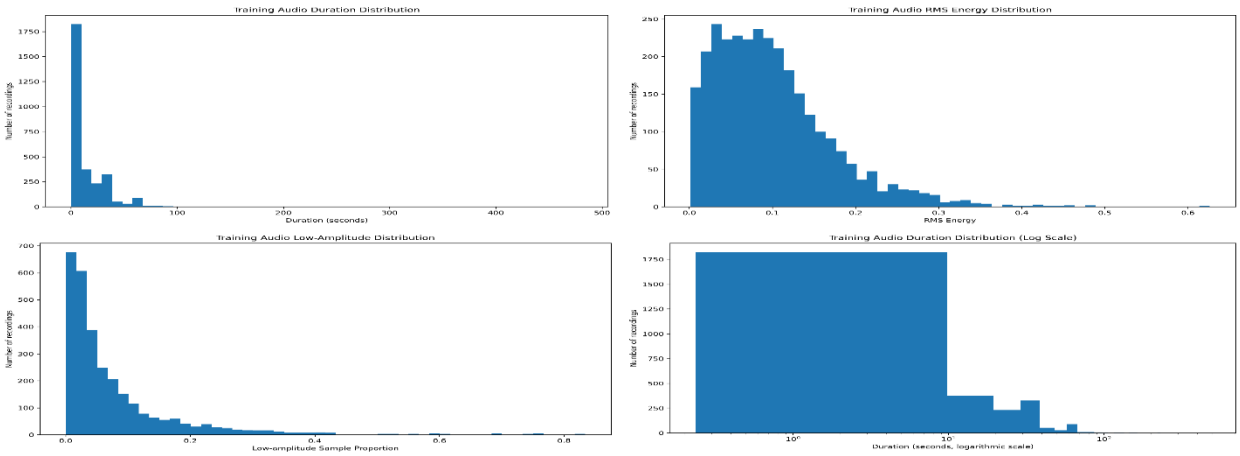


Figure 3. Training-partition audio characteristics, including duration and amplitude.

Data preparation

All model inputs were converted to mono audio and resampled to 32 kHz. Each model input was standardized to 10 seconds. Clips longer than 10 seconds were center cropped, while shorter clips were zero-padded. This made batch processing practical and ensured that the ANN received a consistent input size. The choice also creates a limitation: a relevant sound may occur outside the center portion of a long recording. The application therefore warns users when a long clip is cropped.

I divide this dataset into training, validation and test partitions, with 2,994, 643 and 655 recordings respectively in the original final manifest. One Bells recording was excluded from the final evaluation because it had been used in an earlier compatibility check. The reported final model comparison therefore uses 654 eligible test recordings. All 22 main classes were represented in each partition.

In the duplication check, two pairs of identical files were found. Both were in the training partition. No identified exact duplicate pair crossed the train, validation or test boundaries. However, the dataset does not provide complete original-source identifiers. The split therefore cannot prove that every test

recording is independent of all related source material. The report treats this as a limitation rather than claiming a fully source-independent test.

```
PS G:\My Drive\MSC\AI\Assignment> python src/inspect_dataset.py
Dataset folder: G:\My Drive\MSC\AI\DATASEC\DATASEC
Folder exists: True
Total WAV files: 4292

WAV files by immediate parent folder:
Airplanes: 50
Alarms: 37
Bells: 67
Birds: 69
Brush cutter: 86
Cat fights and moans: 50
Chicken coop: 58
Cicadas: 54
Crickets: 20
Crows: 50
Dog barkings and howlings: 70
Fireworks: 68
Glass breaking: 109
Gunshot: 141
Helicopters: 58
Horn: 57
Jet aircrafts: 103
Lawn mower: 21
Magpies: 21
Music: 245
Olive shaker: 20
Seagulls: 35
Sirens: 68
Thunder: 26
Train: 67
Voices: 1900
Wind turbine: 100
air compressor: 30
car pass-by: 110
car truck idling: 81
```

```
car pass-by: 110
car truck idling: 81
drill: 45
fan: 32
grinder: 38
hairdryer: 32
jackhammer: 63
motorbike idling: 31
motorbike pass-by: 57
saw: 42
truck pass-by: 50
vacuum cleaner: 31

Inventory saved to: G:\My Drive\MSC\AI\Assignment\logs\dataset_inventory.csv
```

```
PS G:\My Drive\MSC\AI\Assignment> python src/check_audio_integrity.py
Checked 500/4292 files
Checked 1000/4292 files
Checked 1500/4292 files
Checked 2000/4292 files
Checked 2500/4292 files
Checked 3000/4292 files
Checked 3500/4292 files
Checked 4000/4292 files

--- AUDIO INTEGRITY SUMMARY ---
Total WAV files found: 4292
Readable WAV files: 4292
Files with read errors: 0
Sample rates (Hz): {44100: 4292}
Channel counts: {1: 4292}
Shortest duration: 0.204 seconds
Longest duration: 481.484 seconds
Mean duration: 14.607 seconds

Detailed results saved to: G:\My Drive\MSC\AI\Assignment\logs\audio_integrity.csv
```

Error costs and choice of evaluation metric

The consequences of classification errors depend on the monitoring task. A false positive occurs when an irrelevant sound is labelled as a class of interest. This may cause an officer to spend time reviewing or retaining a clip that is not useful. A false negative occurs when a relevant sound is assigned to another class. This may cause an event of interest to be missed during the first review. Both types of error matter, and their relative importance may differ between sites.

Macro-F1 was selected as the primary metric because it gives each class equal weight when calculating the average F1 score. This is more suitable than relying on accuracy alone when the class distribution is highly uneven. Accuracy, macro-precision, macro-recall, weighted F1, and per-class results are also reported to give a fuller view of model behavior.

Main findings from EDA and their effect on modelling

Finding	Effect on modelling
Strong class imbalance	Use macro-F1 for model selection and class-weighted cross-entropy for the ANN. Use class-balanced SVM training.
Large variation in clip duration	Use a fixed 10-second input through centre cropping or zero padding, while clearly stating the information loss this may cause.
Incomplete source provenance	Check exact duplicates and keep the split fixed, but describe the remaining risk of source-related leakage.

Question 2: ANN solution design and justification

Model choice

A convolutional neural network was selected because audio can be represented as a time-frequency image. In this project, the waveform is converted into a log-mel spectrogram. Convolutional layers can learn local patterns in this representation, including changes over time and frequency. A pretrained audio network was preferred to training a large model from the beginning because DataSEC has fewer than five thousand recordings.

The project uses the CNN14 architecture from the PANNs family. PANNs were developed by pretraining audio neural networks on a large audio dataset and transferring the learned representations to other audio tasks (Kong et al., 2020). This gives the project a useful starting point, while still requiring the final layers and selected backbone layers to adapt to DataSEC.

Input representation and network structure

The input waveform is converted to a spectrogram using a 1,024-sample analysis window and a hop length of 320 samples. A Hann window is applied, followed by a 64-bin log-mel filter bank covering approximately 50 to 14,000 Hz. Batch normalization is applied to the input representation. Spec Augmentation is used during training to make the model less dependent on a small number of exact time-frequency patterns.

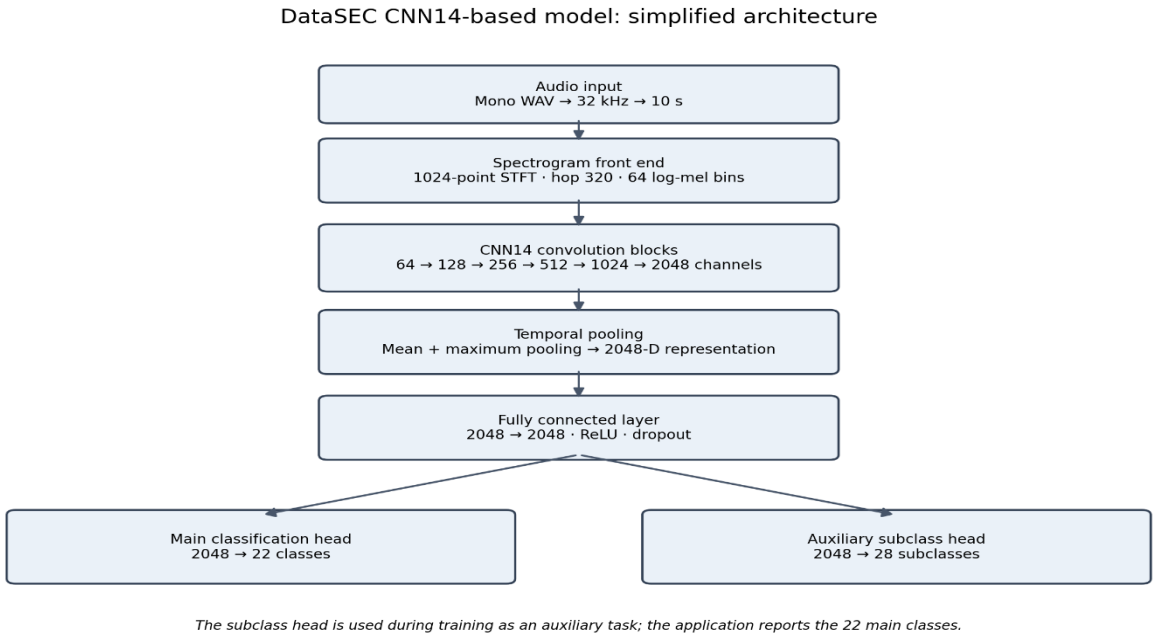


Figure 4. Simplified architecture of the adapted CNN14 model.

Stage	Main configuration	Purpose
Audio preparation	Mono, 32 kHz, 10 seconds	Standardises the waveform length and channel count.
Spectrogram	1024 window, hop 320, 64 mel bins	Represents sound energy over time and frequency.
Convolution blocks	6 blocks; channels 64, 128, 256, 512, 1024, 2048	Learns increasingly complex acoustic patterns.
Temporal pooling	Mean and maximum pooling	Summarises the time dimension into a fixed-size representation.
Fully connected layer	2048 → 2048; ReLU; dropout	Combines learned features before classification.
Main output	2048 → 22	Predicts the main DataSEC class.
Auxiliary output	2048 → 28	Uses subclass labels as an additional training signal.

Each convolution block contains two 3×3 convolution layers with batch normalization and ReLU activation. The first five blocks use 2×2 average pooling, while the final block uses 1×1 pooling. Dropout is applied within the network and around the fully connected layer. The model uses temporal mean and maximum pooling to produce a 2,048-dimensional representation.

The original pretrained network contains an AudioSet output layer with 527 classes. That output is not used for the DataSEC task. Instead, the adapted model adds a 22-class main head and a 28-class auxiliary subclass head. The auxiliary head is used during training; the application displays the main class prediction.

Transfer-learning strategy and parameter count

Most of the pretrained CNN14 backbone was frozen to reduce the number of parameters updated on the relatively small DataSEC dataset. The final convolution block and the fully connected layer were unfrozen, while the earlier convolution blocks remained fixed. The frozen batch-normalization layers were kept in evaluation mode during training. The final model had 60,930,098 trainable parameters in the adapted training configuration. This figure refers to the trainable subset, not the total number of parameters in the full pretrained network.

The choice of fine-tuning depth was also tested. The depth-2 experiment made a larger part of the backbone trainable and increased the trainable parameter count to 75,089,970. Its validation macro-F1 was 0.9041, which was not better than the selected final configuration.

Loss function and optimization

The main classification head uses weighted cross-entropy. The class weights are based on the inverse class frequency and are normalized to have a mean of one. This gives more influence to mistakes on smaller classes. The auxiliary subclass head uses ordinary cross-entropy on examples with a valid subclass label. Subclass logits are masked so that only subclasses belonging to the true main class are considered. The total training loss is the main loss plus the subclass loss, with a subclass loss weight of 1.0.

The final selected configuration used AdamW with a learning rate of 0.0003, batch size 4, weight decay of 0.0001, and seed 123. The model was trained for 15 epochs, with the best checkpoint selected using validation macro-F1. The best validation epoch was epoch 12. The test partition was not used to select the model.

Design assumptions and alternatives

Design decision	Reason for the choice	Alternative considered

Pretrained CNN14	Transfers useful audio features and reduces dependence on a small training set.	Train a smaller CNN from random initialisation; useful as a learning-from-scratch comparison but likely to need more data.
Log-mel spectrogram	Provides a compact representation that is widely used for audio classification.	Raw waveform models; potentially useful but more demanding to train and tune.
Partial fine-tuning	Balances adaptation with the risk of overfitting.	Unfreeze the full backbone or freeze all layers.
Weighted cross-entropy	Addresses the uneven class distribution.	Unweighted cross-entropy may favor frequent classes.
Auxiliary subclass head	Uses the hierarchical labels as an additional learning signal.	Main-class-only training is simpler but ignores available label detail.
Dropout and augmentation	Reduce over-reliance on individual features and improve robustness.	No regularization may increase overfitting.

A numerical performance forecast was not recorded before the experiments. The design expectation was qualitative: a pretrained ANN should benefit from general audio features, while a classical model may remain competitive because the dataset is relatively small. The comparison results are therefore treated as an empirical test of that expectation rather than as confirmation of a pre-registered numerical target.

Why an ANN is appropriate

An ANN is appropriate because environmental sound patterns can be complex and may involve combinations of frequency and timing information. A CNN can learn these patterns directly from spectrograms rather than depending only on manually selected summary features. The classical SVM is retained as a useful baseline because it is faster and easier to inspect. A rule-based system would be unlikely to cover the wide range of natural sound patterns represented by the dataset.

For a larger operational system, a model trained on continuous recordings or a sound-event detection architecture would be more suitable than the current clip classifier. A smaller model or quantized network may also be preferable where inference must run on low-power devices.

Question 3: Model development, evaluation and optimization

Experimental design

The training and validation partitions were used for model development and configuration selection. The final test partition was kept separate from tuning. The final evaluation includes 654 eligible recordings after excluding the one previously exposed Bells clip. The same eligible test examples were used for the CNN14, SVM, and CLAP comparisons. Macro-F1 is the primary comparison metric.

The project reports a majority-class benchmark, a classical SVM, the initial CNN14, the final CNN14, and two CLAP approaches. Confidence intervals were estimated using 10,000 bootstrap samples. Paired bootstrap comparisons were also used to examine the difference between model macro-F1 scores.

Majority-class benchmark

The majority benchmark always predicts the most frequent training class, Voices. It achieved 43.58% accuracy but only 0.0276 macro-F1 on the eligible test set. This illustrates why accuracy alone is not sufficient for this dataset. A model can obtain a seemingly substantial accuracy by repeatedly predicting a common class while failing to recognize the less frequent classes.

Classical machine-learning baseline

The classical baseline uses 50 hand-crafted audio features: the mean and standard deviation of 20 MFCC coefficients, together with mean and standard deviation values for spectral centroid, spectral bandwidth, 85% spectral roll-off, RMS energy and zero-crossing rate. The features are extracted from the standardized 10-second audio input. The classifier is an RBF-kernel support vector machine with standardization and class-balanced training.

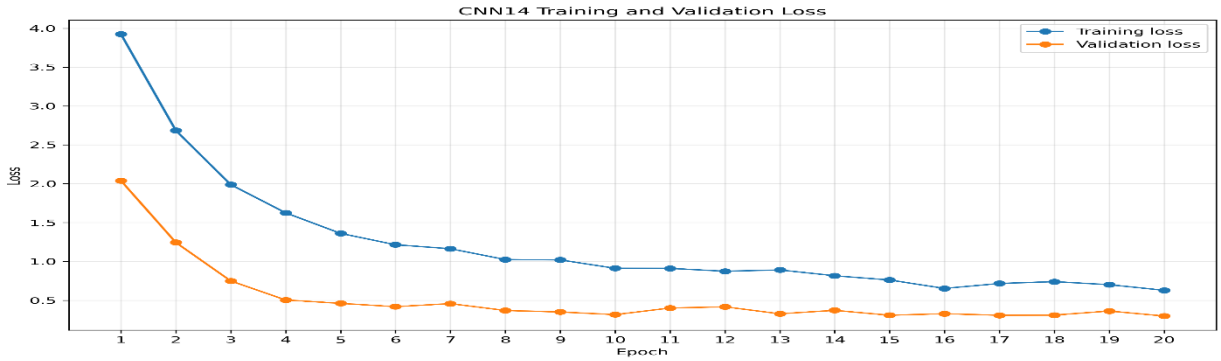
The initial SVM configuration used C=1 and gamma='scale'. A separate validation-only search examined six combinations using three-fold Stratified Group KFold validation: C values of 0.1, 1, and 10, combined with gamma values 'scale' and 0.01. The selected configuration was C=10 and gamma=0.01, with a mean cross-validation macro-F1 of 0.7332 and validation macro-F1 of 0.7472. The held-out test score reported below is for the original SVM baseline, not a score selected by looking at the test set.

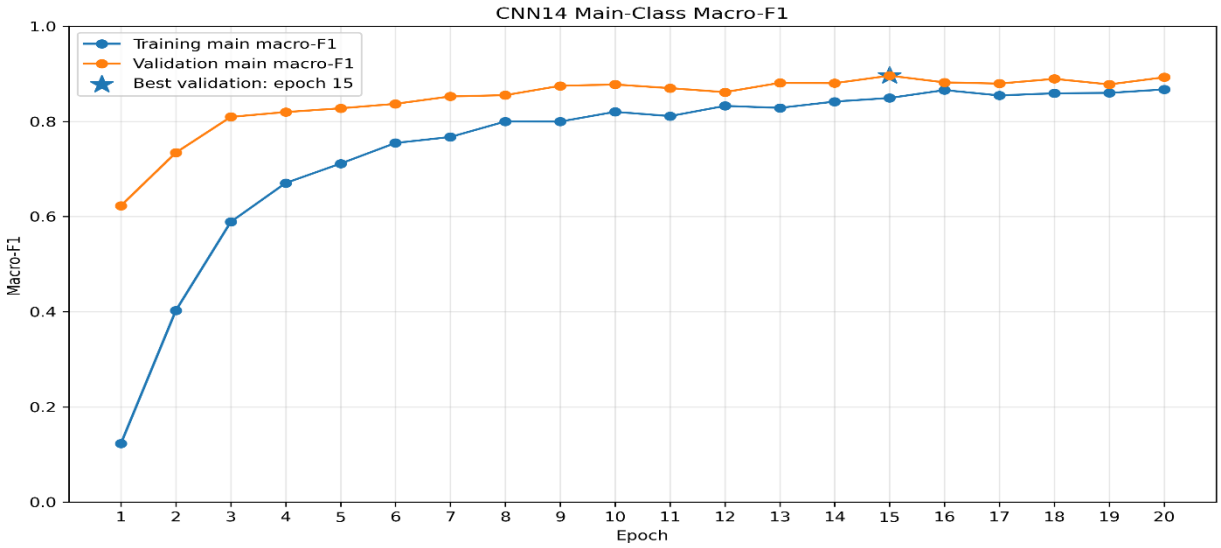
Initial and final ANN

The initial CNN14 was trained for 20 epochs using the initial configuration. Its best validation macro-F1 was 0.8961 at epoch 15. The final selected CNN14 used the AdamW configuration with seed 123 and achieved validation macro-F1 of 0.9116 at epoch 12. Three final seeds were evaluated on the

validation partition: 42, 123, and 456. Their validation macro-F1 values were 0.9070, 0.9116 and 0.9103, respectively. The mean was approximately 0.9096 with a standard deviation of 0.0023.

The initial training history showed a large improvement during the early epochs. Training macro-F1 increased from approximately 0.1229 in epoch 1 to 0.8672 in epoch 20. Validation macro-F1 increased from approximately 0.6228 to a peak of 0.8961 at epoch 15, then decreased slightly to 0.8927 by epoch 20. This pattern suggests that the model learned useful features, while the small late decline indicates that continuing training did not keep improving validation performance. The best validation checkpoint was therefore retained.





Held-out test results

Model	Accuracy	Macro-precision	Macro-recall	Macro-F1	Weighted-F1
Majority class	0.4358	0.0198	0.0455	0.0276	0.2645
MFCC + spectral SVM	0.8563	0.7683	0.7546	0.7571	0.8582
Initial CNN14	0.9480	0.9118	0.9180	0.9137	0.9481
Final CNN14 (FINAL-123)	0.9572	0.9230	0.9226	0.9203	0.9560

CLAP zero-shot	0.8532	0.8338	0.8704	0.8321	0.8565
CLAP linear probe	0.9709	0.9517	0.9593	0.9534	0.9717

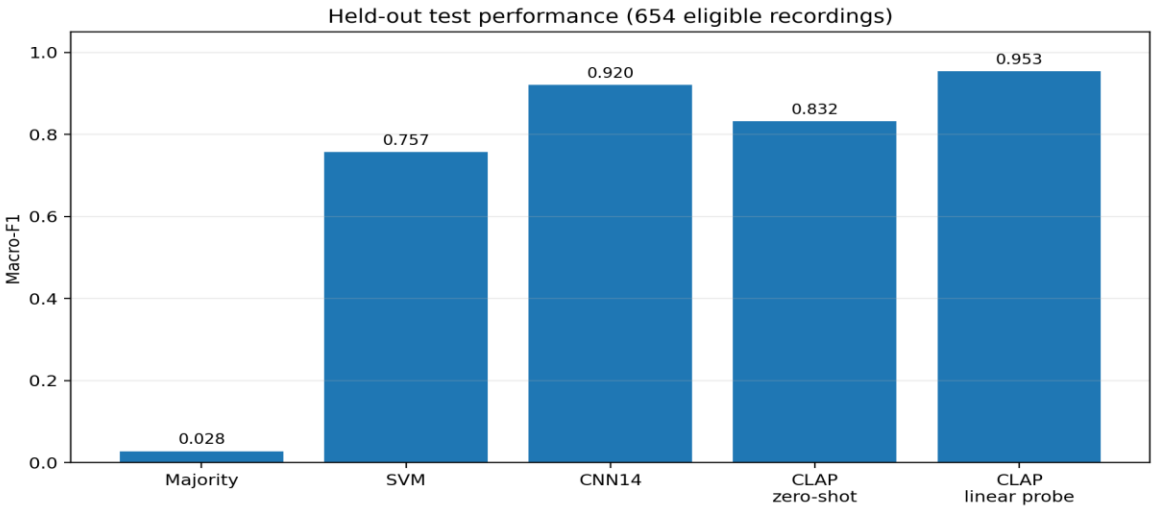


Figure 5. Macro-F1 comparison on the same 654 eligible test recordings.

The CLAP linear probe achieved the highest test macro-F1 0.9534, followed by the final CNN14 0.9203, the initial CNN14 0.9137, CLAP zero-shot 0.8321, and the SVM 0.7571. The majority baseline was much lower at 0.0276. The final CNN14 improved on the initial CNN14 by approximately 0.0065 macro-F1. This is a modest improvement, but it is consistent with the better validation result.

The CLAP linear probe performed best in this experiment. This does not make the CNN14 result unimportant: the CNN14 is the specialist ANN developed for the coursework and can be packaged as a local inference model. However, the comparison shows that the pretrained CLAP representation was highly useful for this dataset. The CLAP zero-shot result was lower than its linear-probe result, indicating that using the audio-text model without fitting a classifier to the DataSEC labels was less effective than adapting its embeddings with a supervised linear model.

Confidence intervals and paired comparisons

Model	Bootstrap 95% interval for macro-F1
CLAP linear probe	0.9340–0.9722
Final CNN14	0.8929–0.9453
CLAP zero-shot	0.8030–0.8603
SVM	0.7143–0.7945

The paired bootstrap difference in macro-F1 between CLAP linear probe and final CNN14 was 0.0332, with a 95% interval from 0.0046 to 0.0633. Since this interval does not include zero, the observed macro-F1 advantage for CLAP linear probe is supported by this paired analysis. The paired accuracy difference was 0.0138, with an interval from -0.0015 to 0.0291, which includes zero. This means the evidence for a difference in accuracy is less clear than the evidence for a difference in macro-F1.

These intervals describe uncertainty across resampled test examples. They do not remove the limitation caused by incomplete source provenance, and they should not be interpreted as proof that the test set represents every real-world monitoring location.

Hyperparameter optimization and optimizer comparison

The optimization experiments examined learning rate, batch size, fine-tuning depth, and optimizer choice. Ten runs were recorded, including five configurations that performed less well than the selected final model. The validation partition was used for these comparisons, and the experiment metadata recorded that the test partition was not accessed.

Run	Optimizer/setting	Validation macro-F1	Training time (s)
LR-LOW	AdamW, lr 0.00003, batch 4	0.8257	5165.84
LR-HIGH	AdamW, lr 0.0003, batch 4, 10 epochs	0.9041	5088.86
BATCH-2	AdamW, batch 2	0.8634	5984.76
BATCH-5	AdamW, batch 5	0.8791	6312.48
DEPTH-2	AdamW, deeper fine-tuning	0.9041	4149.74
SGD-MOM	SGD with momentum	0.7534	4033.48
SGD-LR-HIGH	SGD, lr 0.001	0.6940	3770.96
FINAL-42	AdamW, seed 42	0.9070	5901.81
FINAL-123	AdamW, seed 123	0.9116	6151.17
FINAL-456	AdamW, seed 456	0.9103	7670.08

The learning-rate experiments showed that the lower tested learning rate did not adapt the model as effectively within the available training period. The larger batch settings also produced lower validation macro-F1 than the selected batch size of four. Unfreezing a second convolution block increased the number of trainable parameters but did not improve validation performance. In the tested settings, AdamW performed better than SGD with momentum. This is a conclusion about the configurations tested here, not a claim that AdamW is always better for audio classification.

At least two underperforming approaches were examined. First, the low learning-rate run reached only 0.8257 validation macro-F1, suggesting that the chosen rate was too low for effective adaptation in the fixed training budget. Second, the SGD runs achieved 0.7534 and 0.6940, indicating that these tested SGD settings were not suitable for this fine-tuning setup. The larger batch runs and deeper unfreezing

also failed to improve the final configuration. These were completed experiments rather than software crashes.

Training time and inference latency

The final CNN14 training run took 6,151.17 seconds. The training time for the initial CNN14 and the original SVM was not recorded in the available experiment metadata. These values are therefore reported as not recorded rather than estimated after the event. The tuned SVM fit time was 0.1475 seconds, but this excludes feature extraction and is not directly comparable with the end-to-end neural-network training time.

Model/timing stage	Mean latency	Median	95th percentile
CNN14 model forward pass	38.50 ms	28.45 ms	77.36 ms
CNN14 audio-to-prediction	145.01 ms	88.16 ms	429.67 ms
SVM feature extraction	8.45 ms	8.06 ms	11.50 ms
SVM prediction only	0.97 ms	0.93 ms	1.26 ms
SVM feature + prediction	9.41 ms	9.04 ms	12.66 ms
SVM audio-to-prediction	115.92 ms	63.35 ms	368.54 ms
CLAP audio embedding API call	179.86 ms	141.37 ms	432.87 ms

The timing measurements were made on the project computer using an NVIDIA RTX 2050 GPU. The stages are not identical: some values include audio preparation, while others measure only the model call or feature extraction. The CLAP embedding time is an API-call measurement and does not include the later CPU conversion step. The CLAP model also required approximately 197.5 seconds for one-time loading. For this reason, the latency table should be read as a description of measured stages, not as a perfectly controlled speed ranking.

Error analysis

The confusion matrices and class reports should be used to identify which sound classes are most often confused. Overall scores show strong performance for the final CNN14, but the macro-F1 result is lower than accuracy, which indicates that performance is not equally strong across all classes. The class-level reports are important because they show whether smaller classes are being recognized as reliably as larger classes.

Limitations of the evaluation

The fixed split and exact-duplicate check reduce some forms of leakage, but original recording-source identifiers are not available for every clip. Related source recordings may therefore appear in more than one partition without being detected as exact duplicates. The reported results should be interpreted as performance on the chosen split rather than a definitive estimate of performance on completely independent recording sources. In addition, the test partition was evaluated after model selection and was not used for tuning, except that one Bells clip was removed because it had been used in an earlier compatibility check.

Question 4: Application development and deployment

Application overview

A Streamlit application was developed to demonstrate how the final CNN14 could be used by a potential end user. The user uploads a WAV file, and the application prepares the audio, runs the model, and displays the predicted main class with the five highest class scores. It also presents the uncertainty output and a Grad-CAM explanation. The application is designed as a demonstration of a decision-support workflow rather than a production environmental monitoring service.

The application loads the FINAL-123 checkpoint and the saved calibration and conformal settings. The model is loaded in a cached form to avoid repeating the loading process for every upload. The audio is converted to mono and resampled to 32 kHz. Clips longer than 10 seconds are center-cropped and shorter clips are zero-padded.

Prediction and uncertainty display

The application displays the model's predicted class and class scores. These scores should not be described as guaranteed probabilities of correctness. Temperature scaling was applied using a validation-derived temperature of approximately 0.7807. Calibration is useful because a model can be accurate but still produce confidence values that do not match its actual correctness rate (Guo et al., 2017).

A conformal prediction threshold was also estimated from calibration data. The saved threshold was approximately 0.4059, using the nonconformity score 1 minus the predicted class probability. Under the implemented rule, a class is included when its probability is at least approximately 0.5941. The application can therefore show a set of possible classes or defer when the set is empty or otherwise unsuitable for an automatic suggestion. The empirical test coverage was approximately 91.59%, with a mean set size of 0.94.

These empirical results should not be interpreted as a guarantee of coverage on future recordings. Formal conformal guarantees depend on assumptions about the relationship between calibration and future examples. If the deployment audio differs from DataSEC, coverage may change. The current threshold is therefore a useful review signal, not a safety guarantee.

Input validation and practical tests

Test input	Observed behavior	Interpretation
Bells-0005.wav	Correct prediction; 30.97-second input centre-cropped to 10 seconds; singleton set.	Normal longer-file handling.
Birds-0011.wav	Correct prediction; 5-second input zero-padded; high score.	Normal short-file handling.
Glass breaking-0005.wav	Correct prediction; 2.59-second input zero-padded.	Normal short-file handling.
Stereo version of Glass breaking sample	Converted to mono; prediction remained consistent.	Channel handling test.
Silence	Rejected by the application.	Basic invalid-input protection.
Empty or corrupt audio	Rejected by the application.	File-reading and empty-input checks.
White noise	Predicted Vacuum cleaner fan and hairdryer with a high score; not reliably rejected.	OOD limitation.
Running water recording	Produced an empty conformal set and was deferred.	Useful uncertainty behavior on this example.
Keyboard typing	Predicted Vehicle idling with a high score; not reliably rejected.	OOD limitation.

The tests show that the application handles common input problems such as silence, empty files, and unreadable audio. It also handles mono conversion, short clips and longer clips. However, the unfamiliar white-noise and keyboard examples show that a high score or a singleton prediction set does not always mean that the input belongs to a known DataSEC class. The running-water example was deferred, but the other unfamiliar examples were not. OOD rejection is therefore incomplete.

Deployment suitability and limitations

The application is suitable as a local coursework prototype. It demonstrates the main prediction workflow without requiring an external inference service for the CNN14. It is not yet suitable for unsupervised operational use. It has not been tested across a wide range of recording devices, outdoor conditions, or monitoring sites. It also does not process long continuous recordings as a sequence of events. An operational version would need a stronger OOD detector, broader field testing, access controls, audit logs, and a clear process for human review.

Question 5: Explainable AI and model interpretation

Explanation methods considered

The project compares three explanation methods with different strengths. Grad-CAM uses gradients from a convolutional layer to create a time-frequency map for a selected class. LIME is model-agnostic and estimates how changes to parts of an input affect a particular prediction. Occlusion examines the change in the model score when selected time or frequency regions are masked. The methods were selected to provide more than one view of the model's behavior.

Method	Type	What it shows	Main limitation
Grad-CAM	Architecture-specific	Highlights time-frequency regions associated with the selected class score.	Coarse map; highlighted regions are not proof of causal reasoning.
LIME	Model-agnostic	Approximates the model locally and estimates which input regions influence the prediction.	Can be computationally expensive and sensitive to perturbation settings.
Occlusion	Perturbation-based	Measures how the selected score changes when a time or frequency region is masked.	Results depend on the masking baseline and may create unrealistic audio.

Grad-CAM was chosen for the application because it can be produced from the CNN's convolutional features and displayed as a time-frequency heatmap. The other methods were retained for comparison and evaluation. Grad-CAM was originally introduced as a way to produce visual explanations from gradients flowing into a convolutional layer (Selvaraju et al., 2017). LIME was introduced as a model-agnostic method that fits a simple local explanation around an individual prediction (Ribeiro et al., 2016).

Grad-CAM error examples

Five incorrect predictions were selected for Grad-CAM inspection. Three examples are discussed here in more detail. The heatmaps show which time-frequency areas are associated with the predicted class score. They do not establish that the model has identified the physical cause of the sound.

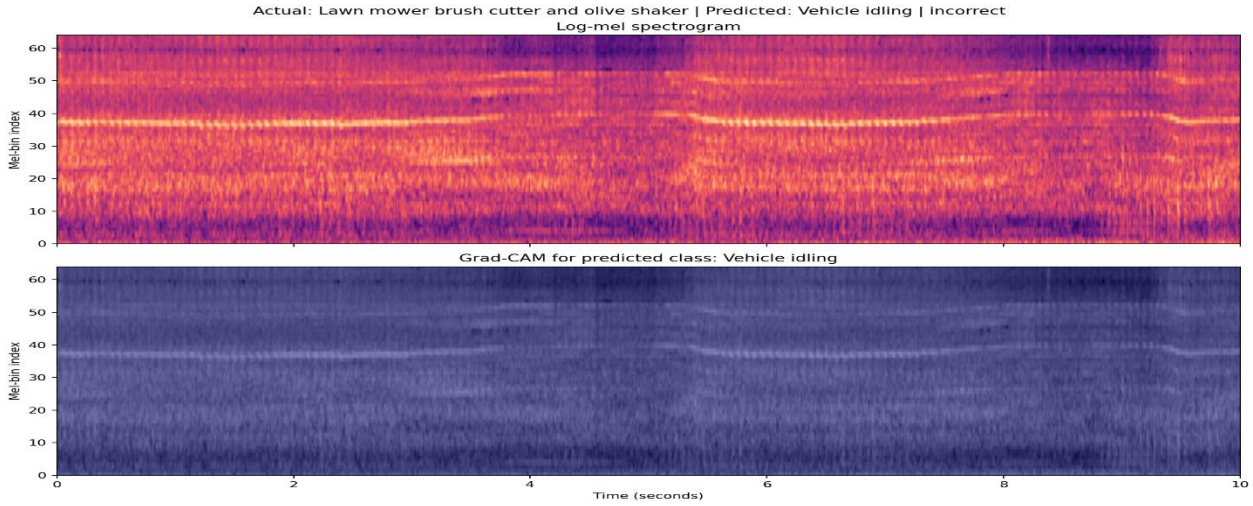


Figure 6. Grad-CAM for an olive shaker recording incorrectly predicted as Vehicle idling.

Above heatmap represent the olive shaker recording appeared almost uniform across the 10-second input, with no clearly localized high-activation area. This makes the explanation difficult to interpret. It does not show a strong time-frequency region that can be linked confidently to the predicted class. The result should therefore be treated as an unclear explanation rather than evidence that the model relied on background noise or a particular irrelevant feature.

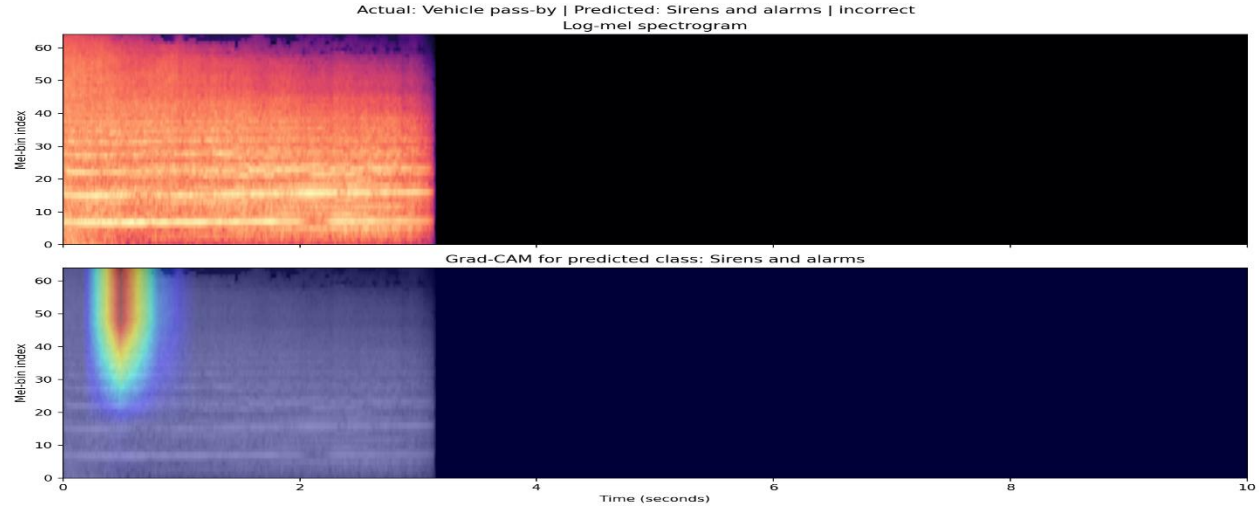


Figure 7. Grad-CAM for a Vehicle pass-by recording incorrectly predicted as Sirens and alarms.

The above heatmap shows a strong, localized vertical activation band near the beginning of the input, approximately within the first second, while most of the remaining time-frequency area had much lower activation. This suggests that the prediction was strongly associated with a short early part of the clip. The input was shorter than the fixed 10-second duration and was zero-padded. The heatmap alone

cannot show whether the active region represents a siren-like feature, a transient from the passing vehicle, or another acoustic detail. It does, however, provide a useful area for a human reviewer to inspect.

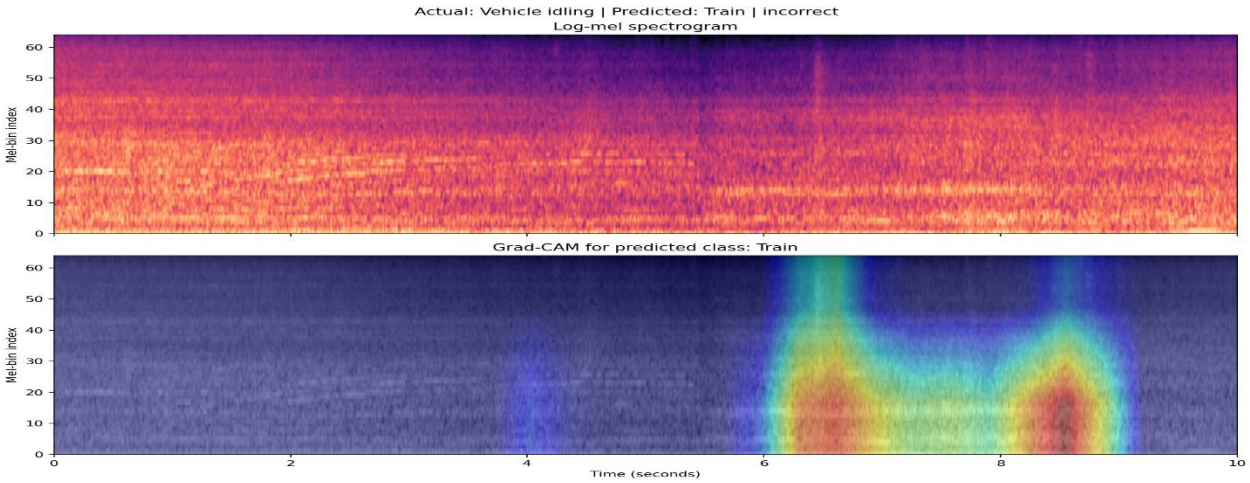


Figure 8. Grad-CAM for a Vehicle idling recording incorrectly predicted as a train.

The above heatmap shows a broader and more diffuse pattern, mainly across the lower and middle frequency bands, with activity spread over the latter part of the input. Unlike the previous example, the activation was not concentrated in one short region. This suggests that the predicted class score was associated with a more distributed pattern in the spectrogram. It is not possible to conclude from the heatmap alone which acoustic feature caused the confusion.

Sanity and perturbation checks

The explanation methods were checked using parameter randomization and perturbation tests. The randomization results showed that the original Grad-CAM maps and maps from randomized network components were not consistently similar. However, some reference maps were constant, and the number of valid comparisons was small. The numerical results should therefore be interpreted cautiously rather than as a complete validation of explanation quality.

Deletion and insertion tests gave mixed results. The mean deletion AUC for Grad-CAM was 0.4106, compared with 0.6430 for random deletion. The mean insertion AUC was 0.7955 for Grad-CAM and 0.6630 for random insertion. The Grad-CAM deletion result was better than random deletion under the chosen scoring direction, while the insertion result was also higher than the random comparison. However, the number of usable examples differed between the tests, and the results varied across recordings.

Targeted occlusion tests also showed that the highlighted regions did not always behave as expected. For a correctly classified jackhammer recording, masking the high-importance region reduced the original predicted probability from approximately 0.99995 to 0.99939, while masking a low-

importance region reduced it to approximately 0.99912. The high-importance region therefore did not produce the larger drop in this saturated example. For an incorrectly classified aircraft recording, masking the high-importance region increased the predicted-class probability from approximately 0.5147 to 0.5945. These findings show why a heatmap should not be treated as a direct explanation of the sound or as proof that a particular region caused the prediction.

Overall assessment of explainability

The three explanation methods provide different kinds of information. Grad-CAM is convenient for the application and gives a quick overview of the time-frequency regions associated with a prediction. LIME provides a model-agnostic local explanation, while occlusion gives a direct test of how masking selected regions changes the score. However, the perturbation results are mixed, and some heatmaps are diffuse or difficult to interpret. The explanations should therefore be used to support human inspection, not to justify a prediction as correct.

The final application uses Grad-CAM because it provides a visual output that can be inspected alongside the class scores and uncertainty result. The interface should make clear that the heatmap is an aid to review. It does not show a causal reason for the prediction, and it does not replace listening to the recording.

Question 6: Generative AI/foundation model evaluation

Foundation model selected

The project evaluates LAION-CLAP, an audio-text foundation model that learns a shared representation for audio and natural-language descriptions. CLAP can compare an audio embedding with text embeddings, which allows it to perform zero-shot classification when class descriptions are supplied. It can also provide audio embeddings for a separate classifier. The original CLAP work describes learning audio concepts from natural-language supervision and evaluates both zero-shot and supervised downstream tasks (Elizalde et al., 2022).

CLAP is a foundation model rather than a generative text model. It does not generate a written explanation in this experiment. It was selected because it is designed for audio understanding and therefore provides a direct and relevant comparison for the environmental sound classification task. If the coursework interpretation requires a model that generates new text or audio rather than a pretrained foundation model, this distinction should be confirmed with the lecturer. The present evaluation should not describe CLAP as a generative model.

Model configuration and evaluation

The evaluated checkpoint was LAION-CLAP HTSAT-fused. The local checkpoint was loaded using the laion_clap package. Audio embeddings were 512-dimensional and extracted in batches of 16. Two approaches were tested. The first was zero-shot classification, where the audio embedding was compared with text embeddings for the 22 main classes. The second was a linear probe, where a logistic regression classifier was trained on the CLAP audio embeddings using the DataSEC training partition.

For the zero-shot approach, prompts were refined using validation data only. The final prompt file contained 66 descriptions, with three descriptions for each of the 22 classes. The prompt set was frozen before final test evaluation. For the linear probe, logistic regression used C=1, a maximum of 2,000 iterations, class-balanced weights and the solver. The classifier was fitted using training embeddings, with validation used for development. The final test set was not used to fit the classifier or select the prompts.

Both CLAP approaches were evaluated on the same 654 eligible test recordings used for the CNN14 and SVM comparison. This provides a more direct comparison than evaluating the models on different test examples.

Performance comparison

The CLAP linear probe achieved a macro-F1 of 0.9534 and accuracy of 0.9709. This was the strongest result among the evaluated models. The zero-shot approach achieved a macro-F1 of 0.8321 and accuracy of 0.8532. The difference between these two CLAP approaches shows that the pretrained representation was useful, but fitting a simple classifier to the DataSEC training embeddings improved performance for this label set.

The paired bootstrap comparison found that the CLAP linear probe exceeded the final CNN14 by 0.0332 macro-F1, with a 95% interval from 0.0046 to 0.0633. The result supports the conclusion that CLAP linear probe performed better on this particular held-out split. It does not establish that CLAP will be better on every environmental sound dataset or in every field setting.

Practical comparison: performance, latency, cost, and privacy

Factor	CNN14 specialist model	CLAP foundation model
Test macro-F1	0.9203	0.9534 linear probe; 0.8321 zero-shot
Adaptation	Fine-tuned on DataSEC labels.	Zero-shot uses text prompts; linear probe requires labelled DataSEC embeddings.

Inference setup	Local model inference using the saved CNN14 checkpoint.	Local checkpoint and embedding extraction in this project; no hosted API was required.
Measured latency	Audio-to-prediction mean 145.01 ms.	Embedding API-call mean 179.86 ms; model load about 197.5 s once.
Privacy	Can be run locally; uploaded audio need not be sent to an external service.	Can also be self-hosted locally, but use of a hosted third-party service would require a separate privacy review.
Operational dependence	Depends on local model and software environment.	Depends on checkpoint availability, package compatibility and model licence terms.
Cost	Local compute and maintenance costs.	Local GPU and storage costs; hosted use may add service charges if selected.

The CLAP checkpoint was approximately 1.86 GB, so model storage and loading time are relevant deployment costs. In this project, inference was performed locally, so audio was not sent to an external hosted service. A future hosted deployment would need to review data handling, service terms and any costs. The Python package license should not be treated as proof that the model weights or all pretraining data have identical license terms. Those should be checked separately before commercial use.

The latency figures are not a perfect end-to-end comparison because the timing stages differ. The CLAP timing records the embedding API call and excludes a later CPU conversion step. The CNN14 audio-to-prediction measurement includes a different set of operations. The numbers are therefore useful for describing the measured environment, but not for making a definitive claim that one model is faster in every deployment.

Overall discussion

The results show that transfer learning with CNN14 is an effective approach for the DataSEC classification task. The final model achieved a test macro-F1 of 0.9203, which is substantially higher than the SVM baseline and the majority benchmark. The CLAP linear probe performed better, achieving 0.9534 macro-F1. This is an important result because it shows that a pretrained audio-text representation can be highly effective even when a specialist ANN is developed and tuned for the same task.

The system also demonstrates a practical workflow. It accepts audio, applies consistent preprocessing, displays a predicted class and score, and provides an uncertainty result and Grad-CAM heatmap. The application can reject empty, corrupt and silent inputs. It can also defer some unfamiliar recordings.

However, the OOD tests show that unfamiliar sounds are not always rejected. A confident prediction may still be wrong when the input is outside the training distribution.

The explain ability experiments add useful information but also show that visual explanations have limits. Some incorrect predictions produced localized heatmaps, while others produced diffuse patterns. The deletion, insertion and targeted occlusion results were mixed. The system should therefore present explanations as review aids rather than evidence that a model has understood the physical sound source.

The main evaluation limitation is incomplete source provenance. Exact duplicates were checked, but the split cannot guarantee independence between original recording sources. This may affect how well the reported test scores transfer to new recording environments. The fixed-length input also means that a relevant event outside the center of a long clip may be missed. These limitations should be addressed in a future version using source identifiers, longer-context processing and field recordings.

Limitations and future innovation

CLAP has two main advantages in this project. It provides a strong pretrained representation, and the zero-shot approach allows class descriptions to be changed without retraining the audio encoder. However, the zero-shot result was lower than the linear probe, and prompt wording can influence the outcome. The linear probe also requires labelled examples and a fitted classifier, so it is not a completely training-free solution.

A realistic future extension would be to use CLAP as an open-vocabulary support tool. An officer could enter a short description of a sound of interest, and the system could compare the recording with that description to suggest possible matches. The specialist CNN14 could continue to provide the fixed 22-class result, while CLAP could help identify sounds that do not fit neatly into the current taxonomy. Any such suggestions would need human review and a separate evaluation before operational use.

The project does not claim that CLAP generates verified natural-language explanations. Its role here is audio-text matching and feature extraction. This distinction is important when describing the model and its capabilities.

Conclusion

This project developed an uncertainty-aware and explainable neural network prototype for classifying environmental sounds in the DataSEC dataset. The final CNN14 model achieved a macro-F1 of 0.9203 on 654 eligible held-out test recordings. It performed substantially better than the classical SVM and majority-class benchmark. The LAION-CLAP linear probe achieved the highest macro-F1 of 0.9534, while CLAP zero-shot classification achieved 0.8321.

The project also delivered a Streamlit application with audio preprocessing, class scores, conformal prediction output, and Grad-CAM visualization. Tests showed that the application handled several normal and invalid inputs, but it did not reliably reject every unfamiliar sound. The explanation experiments provided useful examples for review, although their faithfulness checks were mixed and do not support causal claims.

Overall, the project meets the aim of developing and evaluating an ANN-based decision-support prototype. It should not be used as an automatic regulatory or compliance system. Future work should improve source-aware evaluation, test on independent field recordings, strengthen OOD detection and examine a more realistic continuous-recording workflow.

References

Elizalde, B., Deshmukh, S., Ismail, M. A. and Wang, H. (2022). CLAP: Learning Audio Concepts From Natural Language Supervision. Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). https://arxiv.org/abs/2206.04769

Fredianelli, L. et al. (2025). Environmental Noise Dataset for Sound Event Classification and Detection. Scientific Data, 12, 1712. https://doi.org/10.1038/s41597-025-05991-w

Guo, C., Pleiss, G., Sun, Y. and Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. Proceedings of the 34th International Conference on Machine Learning, PMLR 70, pp. 1321–1330. https://proceedings.mlr.press/v70/guo17a.html

Kong, Q., Cao, Y., Iqbal, T., Wang, Y., Wang, W. and Plumbley, M. D. (2020). PANNs: Large-Scale Pretrained Audio Neural Networks for Audio Pattern Recognition. IEEE/ACM Transactions on Audio, Speech, and Language Processing, 28, pp. 2880–2894. https://doi.org/10.1109/TASLP.2020.3030497

Ribeiro, M. T., Singh, S. and Guestrin, C. (2016). 'Why Should I Trust You?': Explaining the Predictions of Any Classifier. Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations, pp. 97–101. https://doi.org/10.18653/v1/N16-3020

Selvaraju, R. R. et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. Proceedings of the IEEE International Conference on Computer Vision (ICCV). https://doi.org/10.1109/ICCV.2017.74

LAION-AI (n.d.). CLAP: Contrastive Language-Audio Pretraining repository and model resources. https://github.com/LAION-AI/CLAP

AI assistance declaration

In this assignment, I used generative AI tools during the preparation of this coursework to support the planning and organization of the report, grammatical improvement and assist with understanding selected technical concepts. Also, AI was used to structure the discussion of the project methodology and experimental findings. And also get the assistance for debug the system bugs. But the project implementation, dataset preparation, model development, training and testing, and result evaluation were carried by me. As a student, I take full responsibility for the accuracy, originality, and integrity of the assignment and the methods, code and results.

OneDrive Link: COMScDS251P-007 - PPGSPerera (ANN Assignment)

GitHub Repository Link: https://github.com/goyumsamuditha/datasec-uncertainty-aware-sound-classification.git